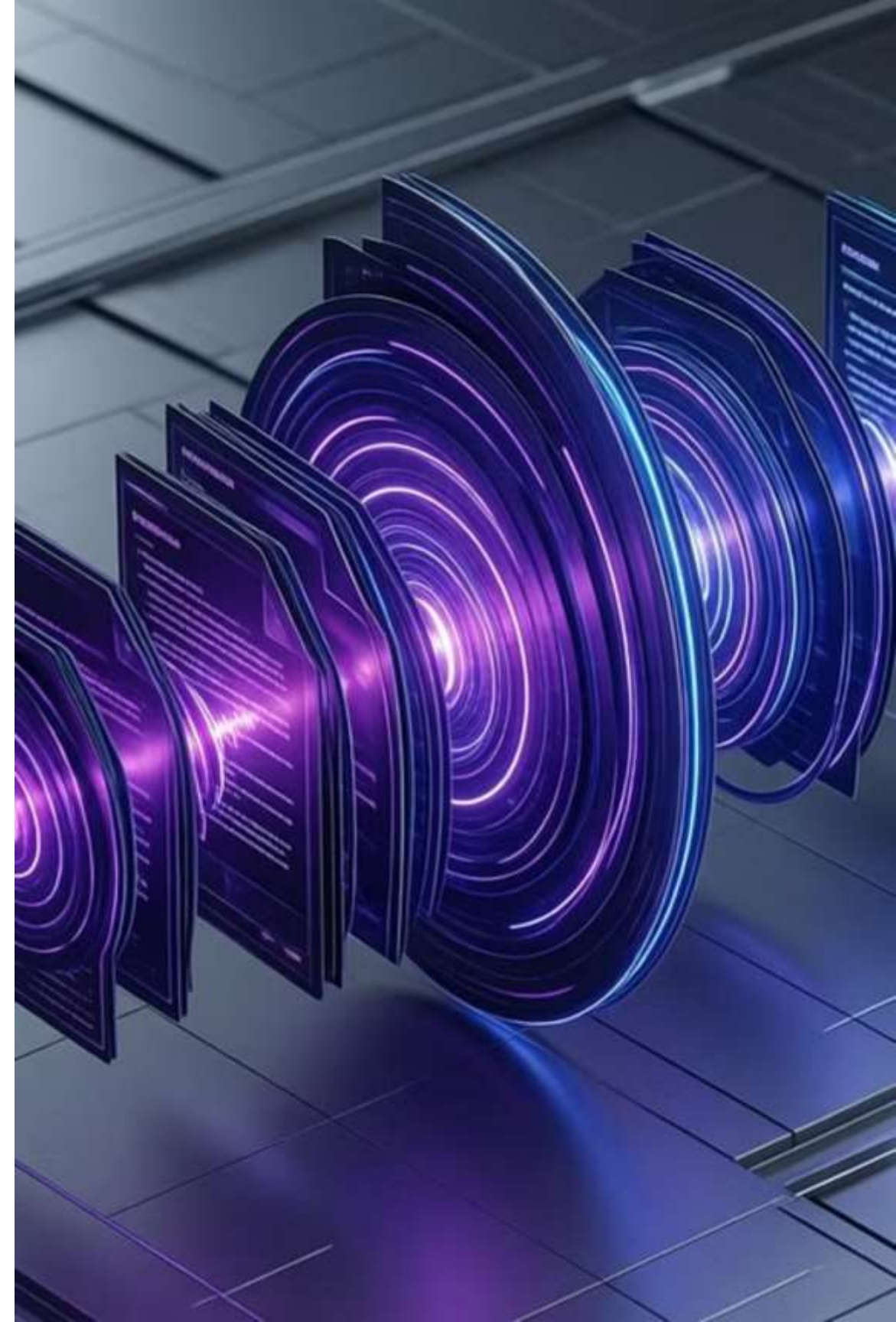


Automated Podcast Transcription & Topic Segmentation

Transforming Unstructured Audio into
Searchable, Structured Knowledge

Presented by: Sugumaran S





The Problem Statement

- **The "Unstructured Data" Crisis:** Podcasts and lectures generate massive amounts of audio that cannot be searched or skimmed.
- **The Efficiency Gap:** Finding a specific 5-minute discussion within a 2-hour episode is tedious and manual.
- **Current Limitations:**
 - Manual transcription is slow and expensive.
 - Existing tools provide a "wall of text" without logical chapter divisions.

Project Objectives

- 1 Precision Transcription:
Achieve near-human accuracy in speech-to-text using **OpenAI Whisper**.
- 2 Intelligent Segmentation:
Use **NLTK TextTiling** to detect natural topic boundaries based on context.
- 3 Smart Summarization:
Deploy **BART** to generate concise abstractive summaries for each topic.
- 4 Keyword Discovery:
Extract semantic keywords using **KeyBERT** for quick scanning.





System Architecture

Frontend Layer:

Streamlit dashboard for user interaction and visualization.

Processing Layer:

Python pipeline managing audio normalization and model inference.

AI Core:

- **Whisper:** ASR (Automatic Speech Recognition).
- **BART:** Abstractive Summarization.
- **KeyBERT:** Keyword Extraction.

Data Layer:

JSON/CSV storage for structured results.

Project Workflow

1

Ingestion:

User uploads MP3/WAV files.

2

Preprocessing:

Silero VAD removes silence; **Librosa** normalizes audio to 16kHz.

3

Transcription:

Whisper converts audio to timestamped text.

4

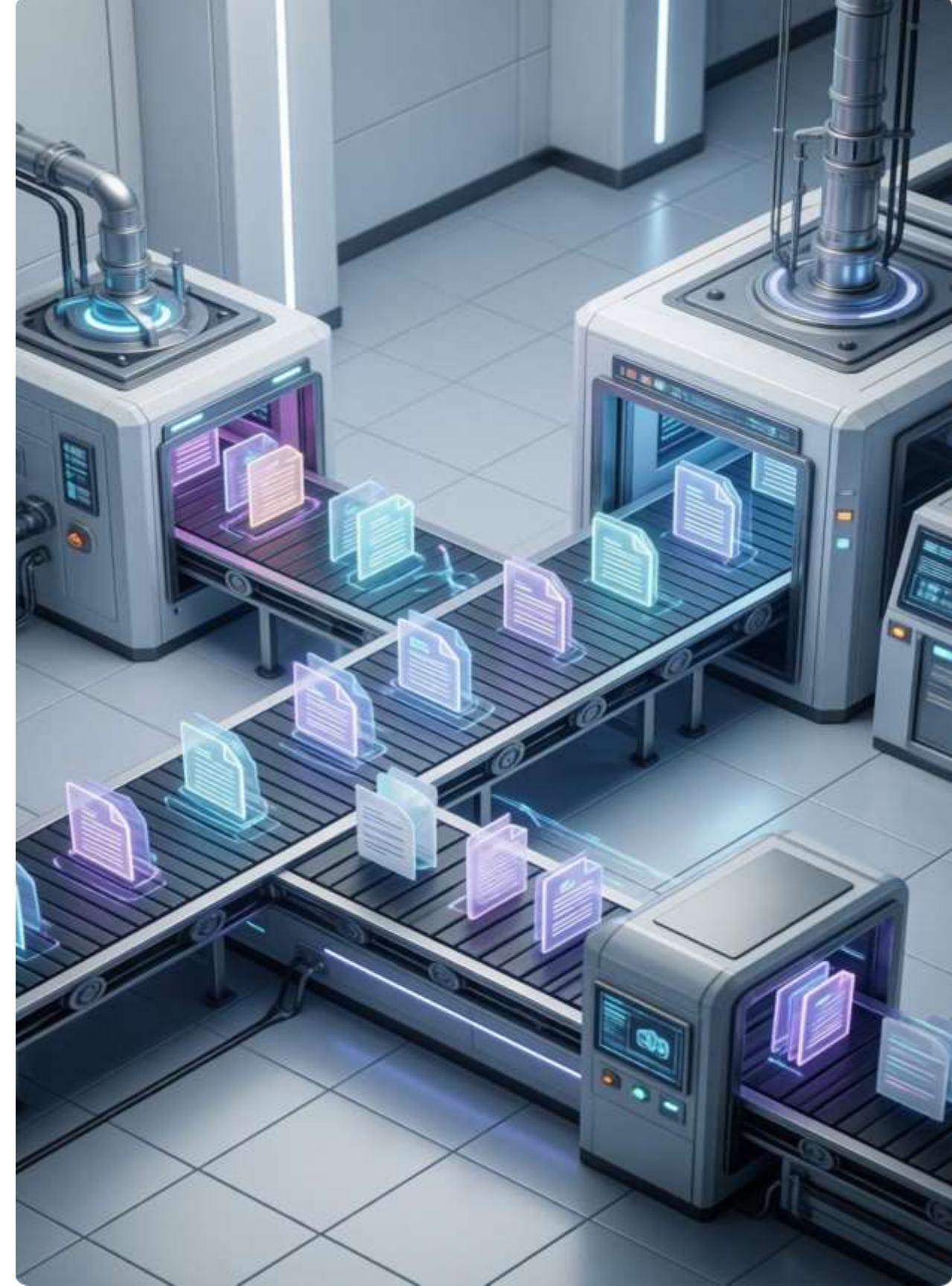
Segmentation:

TextTiling analyzes lexical cohesion to find topic shifts.

5

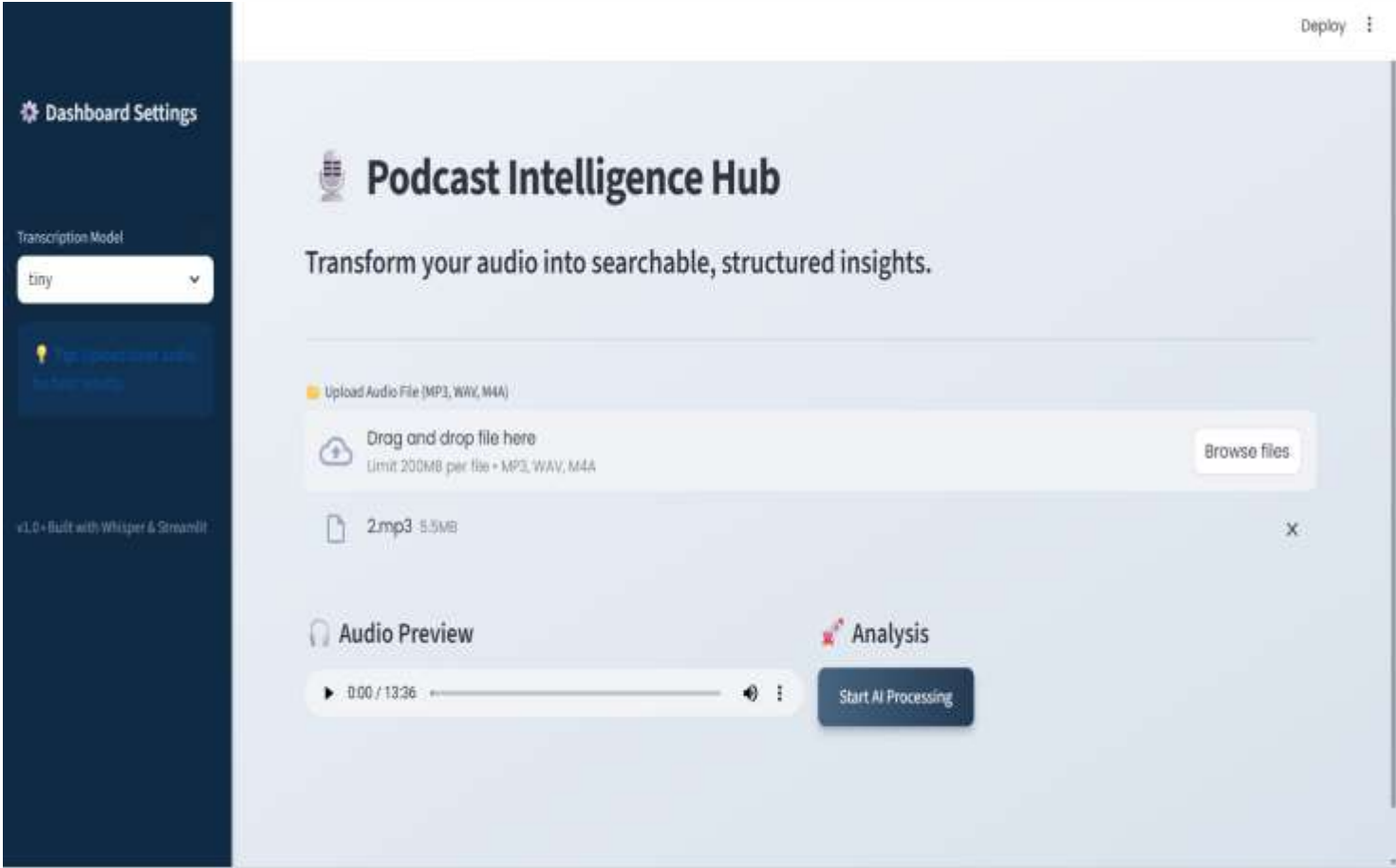
Analysis:

BART summarizes the segment; **KeyBERT** extracts tags.

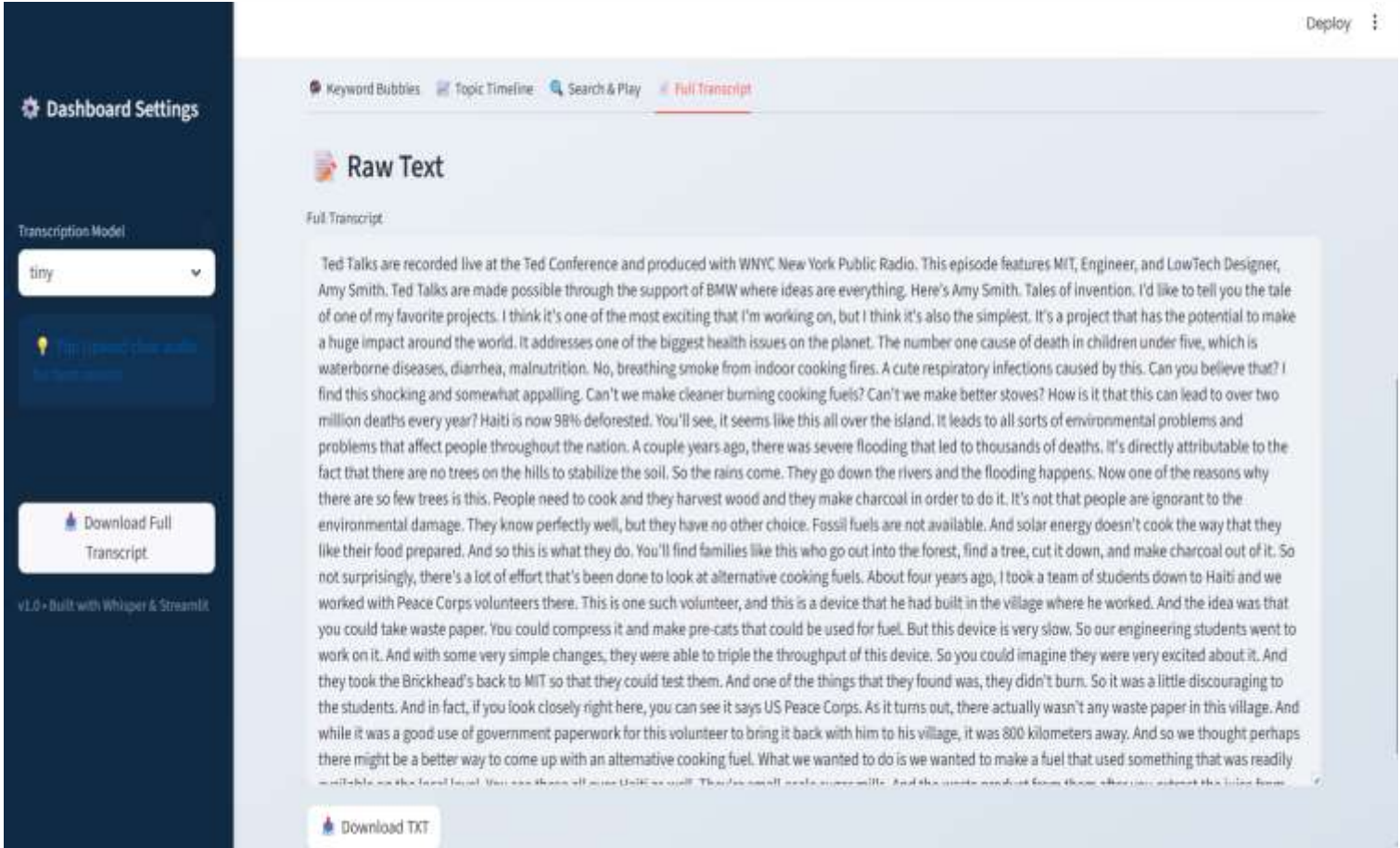


User Interface

Dashboard



Full Transcript



The screenshot displays the 'Semantic Search' feature within the OpenAI Playground. The interface includes a sidebar on the left with navigation links: 'Dashboard Settings', 'Download Full Transcript', and 'API-Only with Offpeak Discount'. The main content area shows a search bar with the query 'e.g. "Digital technology"', a list of 14 results, and a sidebar with navigation options: 'Dashboard Settings', 'Download Full Transcript', and 'API-Only with Offpeak Discount'.

Result ID	Result Text
0000	Talks
0001	Make
0002	Carroll
0003	Super
0004	Wanted
0005	Carroll
0006	Well
0007	Cherwell
0008	Autobly
0009	Cherwell
0010	Thak
0011	Thak

[illegible]



Tech Stack

Core Language:

- Python 3.9+

Audio Processing:

- **Silero VAD:** Enterprise-grade silence removal.
- **Librosa:** Signal processing & normalization.
- **FFmpeg:** Format conversion.

AI Models:

- **OpenAI Whisper:** Transformer-based ASR.
- **BART (HuggingFace):** Abstractive Summarization.
- **KeyBERT:** Semantic Keyword Extraction.
- **NLTK:** TextTiling Algorithm.



Key Features



Automated Timestamping:

Every segment is mapped to precise audio start/end times.



Abstractive Summaries:

Users get a 3-sentence overview of each topic (via BART).



Semantic Tagging:

KeyBERT extracts themes based on meaning, not just word count.



Interactive Navigation:

Click a topic title to jump to that part of the audio.

Performance & Limitations

Performance:

- **Silero VAD** reduced processing time by ~15% by removing silence.
- **Whisper Base** offered the best balance of speed vs. accuracy.

Limitations:

- **Memory Usage:** BART and Whisper models are RAM-intensive.
- **Speaker Diarization:** The system currently does not distinguish between Speaker A and Speaker B.





Conclusion

- **Success:** Built a fully functional "Audio Intelligence" pipeline.
- **Impact:** Reduces content consumption time by transforming linear audio into structured data.
- **Scalability:** The modular design allows easy swapping of models (e.g., upgrading BART to GPT-4).
- **Final Word:** A practical implementation of modern AI to solve a real-world unstructured data problem.

Thank You
Q/A